

How People Use AI Coding Assistants and Software Engineering Agents

A Demand-Side Systematic Mapping Study

Boon Sing Thia

School of Computing and Information Systems, Singapore Management University

bs.thia.2024@smu.edu.sg

Artificial intelligence for software engineering (AI-SE) has moved from inline code completion toward agentic systems that can plan, invoke tools, execute tasks, and iterate across development workflows. Recent surveys describe this transition mainly from the supply side, cataloging agent architectures, capabilities, and evaluation methods [43, 68, 70]. Less is known about the demand side: where empirical studies show software engineers, students, and citizen developers using AI coding assistants, software engineering agents, and AI-augmented low-code or no-code platforms. This systematic mapping study addresses that gap by analyzing 640 primary empirical papers published between 2022 and the April 2026 search cutoff. The study makes three contributions. First, it maps the empirical AI-SE literature across five facets: contribution type, research method, population and context, SDLC activity, and tool profile. Second, it derives a five-mode Human-AI Interaction Mode Taxonomy organized along a delegation-depth axis, from inline completion to delegated task execution. Third, it cross-tabulates 19 supply-side capabilities against nine SDLC activities, identifying 42 low-evidence gaps and 12 empty intersections. The dominant pattern is a supply-demand asymmetry: capability inventories are broad and fast-moving, but empirical evidence remains concentrated around pro-code tools and coding-centered activities, with thinner coverage of requirements, design, documentation, CI/CD, project management, low-code, and no-code contexts. The resulting map provides a reusable foundation for future empirical research, tool design, organizational adoption, and AI-SE education. To support transparency and computational reproducibility, the complete research repository for this study is available at <https://github.com/alphathia/erp2-submit.git>.

CCS CONCEPTS • Software engineering • Software development techniques • Human-centered computing • Artificial intelligence Agents

Additional Keywords and Phrases: software engineering agents, low-code development agents, no-code development agents, systematic mapping study, AI-assisted coding, human-AI interaction, usage patterns, agentic software engineering

1 INTRODUCTION

Since 2021, AI in software engineering has moved from a single-line autocompletion paradigm popularized by GitHub Copilot [21] to a multi-step agentic paradigm. Today, commercial systems such as Cursor, Windsurf, Claude Code and Devin can autonomously plan, invoke external tools, execute, and iterate across the software development lifecycle (SDLC). The distinction matters: agentic AI is not synonymous with generative AI. A generative assistant returns one static response per prompt; an agentic system runs an active reasoning loop, uses tools, and executes multi-step plans autonomously. This fundamentally changes the human’s role by shifting the workload from continuous, granular review to higher-level checkpoint design, trust calibration, and error management.

Three recent surveys examine the supply side of this technological transition. As these surveys detail the capabilities and architecture of AI systems in software engineering, there remains an opportunity to synthesize broad empirical evidence on how professional software engineers, students, and citizen developers use them in practice.

As a result, a demand-side gap remains in the current literature. As seen with previous shifts, such as cloud computing [53], DevOps [47], low-code and no-code development [1], technical capabilities often differ from how it is practically applied. A supply-side only perspective cannot address the critical questions of practitioners, educators, and tool-builders regarding the distribution of demand: specifically, where empirical evidence of usage has accumulated, which interaction patterns are being reported, and which intersections of capability and activity remain under-evidenced. Therefore, this research aims to address the following research questions:

- RQ1. landscape: What is the landscape of empirical research reporting how software engineers, students, and citizen developers use AI coding assistants, SE agents, and AI-augmented low-code and no-code platforms?
- RQ2. Interaction modes. What human-AI interaction modes are reported in the empirical literature, and how do they vary across SDLC activities, agent / tool profiles, and population and context?
- RQ3. Capability-evidence gaps: Where do gaps exist between the capabilities emphasized in recent supply-side surveys and the SDLC activities for which empirical evidence has accumulated?

Answering these three research questions deepens our understanding of how AI coding assistants, software engineering agents, and AI-augmented low-code/no-code platforms are used in empirical software engineering research. Recent supply-side surveys have mapped what AI systems can do, including their capabilities, architectures, and evaluation methods [43, 68, 70]. However, less is known about the demand side: how software engineers, students, citizen developers, and other users engage with these systems in practice, across which software development lifecycle activities, and with what forms of human-AI interaction. This study addresses that gap by systematically mapping the empirical literature and by distinguishing between what an AI system can do (**capability**), how a tool exposes that capability (**modality**), and how humans engage with the tool in practice (**interaction mode**).

The three research questions are therefore designed to move from evidence mapping to interaction synthesis, and to gap diagnosis. RQ1 establishes the empirical baseline by identifying where demand-side evidence has accumulated. RQ2 synthesizes reported human-AI interaction patterns into a reusable taxonomy. RQ3 compares empirical usage evidence with the capabilities emphasized in supply-side surveys, identifying where technical claims have stronger or weaker demand-side support. Together, these questions shift the study from describing AI coding assistants and agents to systematically comparing their **supply-side capabilities with demand-side empirical evidence** of use in software engineering.

Taken together, the result is a demand-side systematic mapping study that complements supply-side AI-SE surveys and offers a reusable foundation for future research, tool design, organizational adoption, and education.

2 BACKGROUND AND RELATED WORK

2.1 The supply-side reference surveys

Three recent surveys lay the foundation for this research. Wang et al. [68] review AI agentic programming, classifying systems across behavioral dimensions like tool augmentation and adaptability. A second survey by Wang et al. [70] studies software engineering agents through a perception-memory-action framework, and Otoum & Elkhaili [43] provide a methods-oriented review structured around lifecycle integration and evaluation. All three flag adoption as a critical open question and identify core capabilities like code completion, program repair, and multi-agent collaboration. However, none provides a demand-side taxonomy or cross-tabulates these capabilities against empirical usage. This study addresses these limitations by synthesizing their disparate capability vocabularies into 19 unified identifiers and cross-tabulating them against nine SDLC activities.

2.2 Prior developer-AI interaction taxonomies

Two adjacent works frame the interaction-analysis contribution of this paper. Treude and Gerosa [63] provide a conceptual taxonomy of developer-AI interaction mechanisms at the user-interface level, while Kumar et al. [33] provide fine-grained observational evidence of developer-agent collaboration in an IDE-integrated setting. This study complements both: it derives interaction modes from a 640-paper empirical corpus, organizes them along a delegation-depth axis, and links them to capability-SDLC evidence gaps rather than to UI mechanisms or single-tool chat trajectories.

2.3 Adjacent work on platform paradigms

Two topic-adjacent surveys delimit the platform-paradigm boundary of the review. Ajimati et al. [1] systematically review low-code and no-code development adoption and explicitly foreground citizen-developer contexts, which are part of the inclusion scope but remain sparsely represented in the empirical corpus. Liu et al. [37] survey LLM-based agents for software engineering, overlapping with the Wang surveys on supply-side agent capabilities while extending attention to broader LLM-enabled SE applications. In this study, both papers are used as backward-snowballing seeds, alongside the three primary reference surveys used for capability harmonization, rather than as sources for the formal RQ3 capability vocabulary.

2.4 Methodological anchors

The study uses four methodological anchors. Petersen et al. [48, 49] guide the systematic mapping study design, including the classification scheme, PRISMA flow, and visualization choices. Cruzes and Dybå [12, 13] guide the thematic synthesis used to derive the RQ2 interaction taxonomy and support the separation between extraction-level coding and higher-order synthesis. Wieringa et al. [76] provide the contribution-type vocabulary used for F1 and the distinction between Evaluation Research and Validation Research. Ali and Petersen [4] guide the backward snowballing procedure. To assess trustworthiness, the study uses Lincoln and Guba’s four qualitative-research criteria: credibility, confirmability, dependability, and transferability—as adapted for software engineering by Cruzes and Dybå [12, 13].

2.5 Positioning

This paper is therefore positioned as an early demand-side complement to a subfield already rich in supply-side surveys. It does not compete with the three reference surveys; rather, it maps the empirical distribution of human usage, interaction modes, and capability-use contexts that those surveys do not set out to characterize.

3 METHOD

3.1 Study Design

This study is a systematic mapping study (SMS) [48, 49]. SMS is appropriate here because AI-SE is an emerging subfield in which scoping, classification, and trend identification are more tractable than pooled effect estimation. Execution proceeded through seven phases (see *Figure 1*) - Phase 0: protocol scaffolding, Phase 1: protocol finalization and capability harmonization, Phase 2: search and screening, Phase 3: full-text extraction with concurrent open coding, Phase 4: code consolidation and theme translation, Phase 5: frequency analysis and cross-tabulation, Phase 6: trustworthiness assessment, and Phase 7: manuscript drafting. Every phase produces a tagged artifact set and a commit checkpoint; every borderline classification decision is logged in an append-only decision register with the rule applied and the rationale.

3.2 Protocol scaffolding and finalization

The study protocol was developed in two distinct phases prior to the formal execution of the search.

Phase 0 (Scaffolding) involved the commitment of the initial pre-registered protocol to version control, encompassing the research questions, a SMS design outline, a preliminary codebook, an extraction form template, and six Scopus search-string archetypes.

Phase 1 (Finalization) yielded four primary protocol artifacts that served as the foundational framework for subsequent stages:

- **Five-Facet Codebook:** Operationalizes the F1–F5 vocabularies through formal definitions, positive/negative exemplars, and boundary-case rules.
- **Capability Vocabulary:** A consolidated 19-capability taxonomy that collapses similar capabilities identified across the three primary reference surveys.
- **Extraction Schema:** Defines the per-paper data fields, sample characterization metrics, and capability-annotation formats.
- **Inclusion and Exclusion Criteria (IC/EC):** Establishes the mechanical filters and manual screening heuristics.

Systematic Mapping Study Process

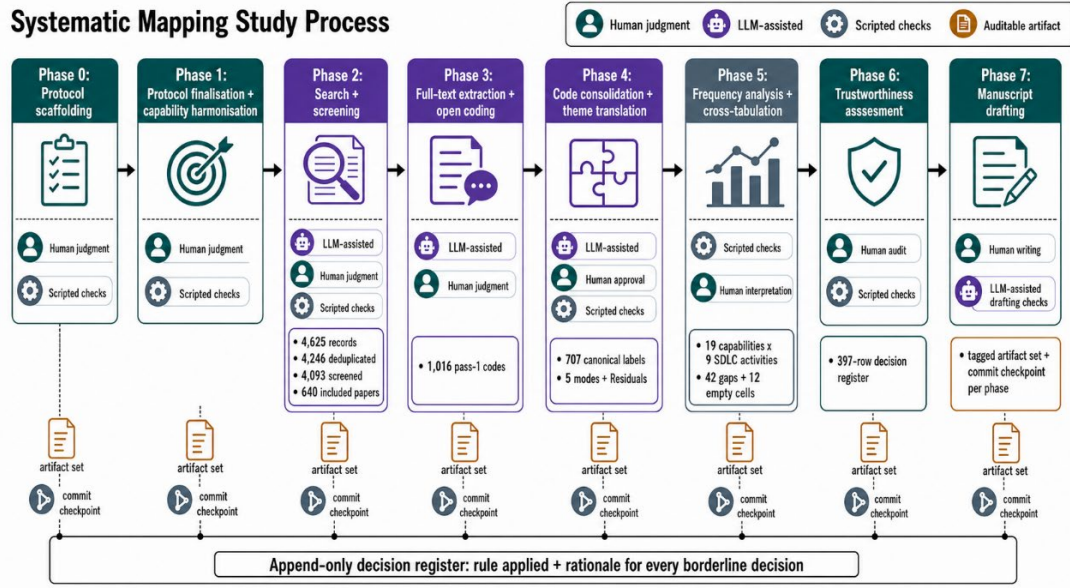


Figure 1: Systematic Mapping Study Process

3.3 Search strategy

The systematic search was conducted through a multi-stage process involving automated database queries and manual citation tracking. To ensure comprehensive coverage, six Scopus search strings were executed in parallel, targeting distinct domains: code-completion tools, conversational assistants, agentic Software Engineering (SE) tools, AI-augmented low-code/no-code platforms, visual/declarative programming, and citizen-developer contexts. The Scopus result set was complemented by a dedicated ACM Digital Library run using ACM-DL query syntax, and supplemented by backward snowballing [4] from the five seed surveys: the three reference surveys [43, 68, 70] plus two topic-adjacent papers [1, 37]. Duplicate overlap between ACM-DL and Scopus was removed at deduplication.

3.4 Inclusion and exclusion criteria; screening

The review protocol established five inclusion criteria (IC) and six exclusion criteria (EC) prior to screening. To be included, papers had to satisfy: (IC1) publication between 2022 and 2026; (IC2) empirical evidence of human interaction with AI software engineering agents, AI coding tools, or AI-enhanced low-code platforms; (IC3) peer-reviewed status; (IC4) English language; and (IC5) a minimum length of four pages. Papers were excluded if they were: (EC1) solution proposals lacking user evidence; (EC2) benchmark-only studies without human participants; (EC3) outside the software engineering domain; (EC4) secondary studies, such as literature reviews; (EC5) short papers, posters, or extended abstracts; or (EC6) records for which neither the full text nor sufficient bibliographic or abstract evidence was available to assess eligibility.

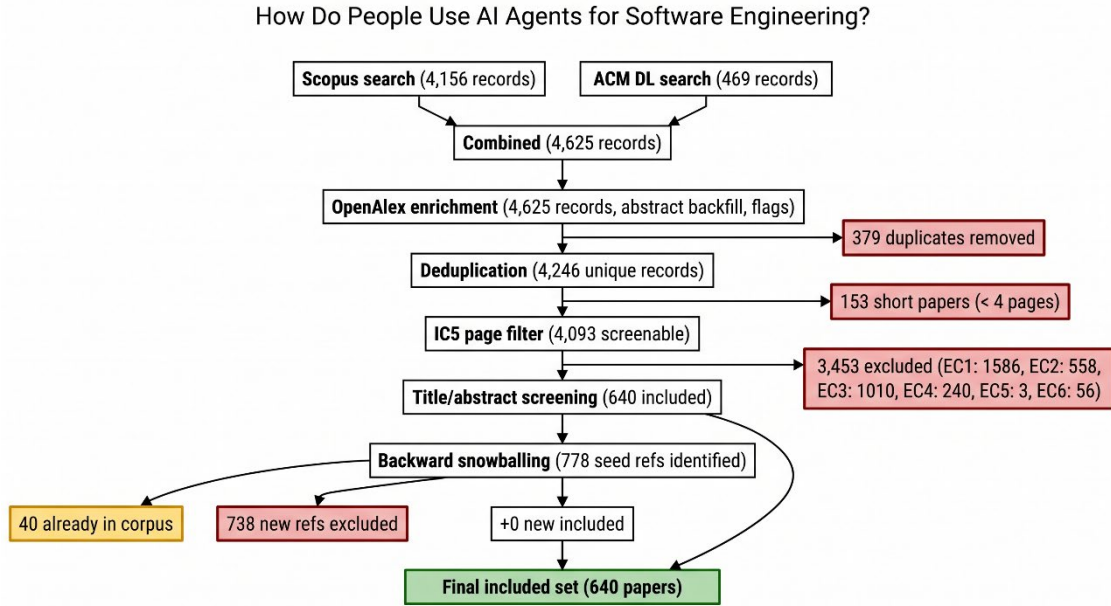


Figure 2: Illustration of the corresponding PRISMA screening flow

Figure 2 illustrates the corresponding PRISMA screening flow. The initial search yielded 4,625 records (4,156 Scopus; 469 ACM DL). After removing 379 duplicates and filtering out 153 short papers (IC5), 4,093 records advanced to title and abstract screening, resulting in 3,452 exclusions and one deferred unresolved record, yielding 640 included papers. Backward snowballing from five seed surveys yielded no new additions, as all relevant references were already captured. The final included corpus comprises 640 papers.

3.5 Classification Scheme

Consistent with established SMS guidance [48], the classification scheme provides the analytic structure for the review. Each included paper was coded across five pre-registered facets (F1-F5). Two additional protocol dimensions, Interaction Mode and Capability Category, were treated as downstream analytic outputs for RQ2 and RQ3 rather than as screening facets. This preserves the methodological distinction between extraction-level description and synthesis-level interpretation [12].

- **F1 - Contribution type** (RQ1). The paper coded using Wieringa et al.'s [76] vocabulary: Evaluation Research, Validation Research, Solution Proposal, Philosophical, Opinion, and Personal Experience. F1 is single-valued per paper, coded at title and abstract level during screening, and revised at full-text read-through when warranted.
- **F2 - Research methodology** (RQ1). The empirical method used to collect or analyze human-interaction evidence. The pre-registered values are 1) Survey, 2) Interview, 3) Case Study, 4) Experiment, 5) Field Study, 6) Mining Study, and 7) Mixed. F2 is single-valued; studies combining multiple methods are coded as Mixed, with component methods recorded in extraction notes.
- **F3 - Population and context** (RQ1, RQ2). The participant group and study setting. Population values are 1) Professional Software Engineer, 2) Student, 3) Citizen Developer, 4) OSS Contributor, and 5) Mixed; context

values are 1) Industry, 2) Education, 3) OSS, 4) Lab, and 5) Mixed. Where no human cohort was identifiable, the field was coded as N/A. Boundary cases involving non-developer end users were mapped to the closest applicable category and logged in the decision register.

- **F4 - SDLC activity** (RQ1, RQ3). The SDLC activity in which the AI tool was used: 1) Requirements, 2) Design, 3) Coding, 4) Testing, 5) Code Review, 6) Debugging, 7) CI/CD, 8) Documentation, or 9) Project Management. F4 is multi-valued because many studies report AI use across multiple activities. Figures using this facet state whether counts refer to distinct papers or paper-activity instances.
- **F5 - Agent / tool profile** (RQ1, RQ2). The AI system studied represented by two sub-facets. Development Paradigm is single-valued: 1) Pro-code, 3) Low-code, or 3) No-code. Interaction Modality is multi-valued: 1) Autocomplete, 2) Conversational, 3) IDE-Integrated, or 4) Autonomous. Multi-valued modality coding preserves the empirical signal from hybrid tools that combine, for example, autocomplete, chat, and IDE integration.

F1 adopts Wieringa et al.'s established contribution-type classification [76]. F2–F5 use study-specific controlled vocabularies with operational definitions, positive and negative examples, and boundary-case rules to support consistent coding. Because both SDLC activity (F4) and tool modality (F5) can be multi-valued, Appendix A.1 reports a ranked full-text cross-tab showing how these two facets combine in the extracted corpus.

We also distinguish three related constructs. Capability describes what an AI system can do, such as code generation, testing, or debugging. Modality describes how the tool presents that capability to users, such as autocomplete, chat, IDE integration, or autonomous execution. Interaction mode describes how users work with the tool in practice, such as accepting inline suggestions, prompting conversationally, reviewing AI output, or delegating a task. This distinction prevents tool features from being conflated with observed human-AI usage patterns.

3.6 Extraction and open coding

Data extraction used a structured schema capturing bibliographic metadata, F1-F5 facet codes, sample characteristics, verbatim human-AI interaction passages, and capability annotations. Capability annotations used a harmonized 19-capability vocabulary derived by merging and deduplicating capability lists from the three core reference surveys.

Extraction followed two modes based on full-text availability. **Mode A** applied to papers for which a retrievable PDF fell within the page-count envelope. The retrieval cascade recovered 258 PDFs; 208 were eligible for full-text extraction, while 50 oversized PDFs were routed to abstract-only extraction. Mode A papers contributed full-text passages, facet codes, and capability annotations.

Mode B applied to the remaining 432 papers: 382 without retrievable PDFs and 50 oversized PDFs. These papers contributed to RQ1 facet coding and, where the abstract contained sufficient evidence, to RQ3 capability annotation. Although Mode B was abstract-only, 82 papers contained quotable interaction passages and therefore also contributed to RQ2. The remaining 350 Mode B papers contributed to RQ1 and RQ3 only.

First-pass open coding generated 1,016 in-vivo and descriptive codes across 290 passage-bearing papers: all 208 Mode A papers plus 82 Mode B papers with interaction evidence in the abstract. This 290-paper subset forms the evidentiary base for the inductive RQ2 synthesis. All results report the applicable denominator, distinguishing full-corpus analyses from passage-based analyses where necessary.

3.7 Analysis

Analysis followed three tracks aligned with the research questions.

RQ1: Evidence landscape. We computed descriptive frequencies across F1-F5 for the full 640-paper corpus. Following SMS conventions [48, 49], results are reported with bubble plots, bar charts, and heatmaps. For multi-valued fields such as SDLC activity and tool modality, each figure states its counting unit, distinguishing distinct-paper counts from paper-activity or paper-modality instances.

RQ2: Interaction modes. We applied thematic synthesis [12, 13] to the 290 passage-bearing papers. First, 1,016 in-vivo and descriptive open codes were generated during extraction. Second, these codes were consolidated into 707 canonical labels using sentence-transformer clustering with a cosine threshold of 0.35; all merges, splits, and renaming required rater approval and were logged. Third, each canonical label was classified into one of five interaction modes or a Residuals category along the pre-committed delegation-depth axis. During synthesis, the provisional eight-mode frame collapsed into five modes: Paired Collaboration folded into Conversational Prompting; Scaffolded Guidance distributed across Conversational Prompting and Delegated Task Execution; and HITL Delegation and Autonomous Orchestration merged into Delegated Task Execution. Mode-layer saturation was assessed using the stopping rule of zero new modes in the final 10% of passage-bearing papers; the final 29 papers introduced no new modes. Mode exemplars were paraphrase-checked against source passages using a 15-word overlap heuristic.

RQ3: Capability-use gaps. We cross-tabulated 19 harmonized capabilities against nine SDLC activities, yielding a 171-cell capability-by-activity matrix. Cell values count distinct papers with both the capability annotation and the SDLC activity. The annotation file contains 776 annotations from 384 papers; after excluding annotations without usable SDLC metadata, 383 papers populate the final matrix. A nonzero cell was classified as a gap when its evidence count was at or below the 25th percentile of the nonzero cell distribution. In this corpus, $P25 = 2.0$; therefore, cells with one or two papers were treated as low-evidence gaps. Zero-evidence cells were reported separately as empty cells, preserving the distinction between under-studied and unobserved intersections. This procedure produced 42 gaps and 12 empty cells. Sensitivity checks across P20-P40 preserved the P25 gap core, with gap counts of 42, 42, 53, and 72 at P20, P25, P33, and P40, respectively.

4 RESULTS

4.1 RQ1 - Evidence landscape

The final corpus comprises 640 studies published between 2022 and 2026, with a pronounced recency skew: 584 of 640 papers (91.3%) appear from 2024 onward. The 2026 count is partial because the search cut-off precedes year end. This distribution is consistent with the post-release inflection in generative-AI tooling, particularly following ChatGPT and GitHub Copilot, and situates the SMS within a rapidly emerging research area. Regarding venue type, 456 papers (71.2%) are conference publications and 184 (28.8%) are journal articles. Disaggregated counts by year, F1 class, and F2 method are reported in Table 1.

F1 - Empirical Evaluation Dominates. Evaluation Research constitutes most of the corpus (459 papers, 71.7%), followed by Validation Research (143, 22.3%). By contrast, Solution Proposals (29, 4.5%), Personal Experience reports (7), and Opinion papers (2) are sparsely represented. The corpus is therefore overwhelmingly oriented toward empirical investigation of AI-enabled software engineering (AI-SE) in context, consistent with the study’s demand-side framing and the explicit exclusion of non-empirical work under ECs 1 and 2.

Table 1: Corpus descriptives (N = 640)

Year	Count	%	F1 Class	Count	%	F2 Method	Count	%
2022	8	1.2%	Evaluation Research	459	71.7%	Experiment	249	38.9%
2023	48	7.5%	Validation Research	143	22.3%	Mixed	139	21.7%
2024	226	35.3%	Solution Proposal	29	4.5%	Case Study	92	14.4%
2025	288	45.0%	Personal Experience	7	1.1%	Mining Study	63	9.8%
2026	70	10.9%	Opinion	2	0.3%	Survey	63	9.8%
						Interview	20	3.1%
						Field Study	14	2.2%

F2 - controlled experiments dominate; qualitative methods are thin. Experimental designs account for the largest share (249 papers, 38.9%), followed by Mixed methods (139, 21.7%), Case Studies (92, 14.4%), Mining Studies (63, 9.8%), and Surveys (63, 9.8%), collectively covering 606 of 640 papers (94.7%). In contrast, Interview-based studies (20, 3.1%) and Field Studies (14, 2.2%) together comprise only 34 papers (Figure 3). This relative scarcity of in situ, first-person qualitative inquiry provides an independent rationale for subsequent research employing such methodologies.

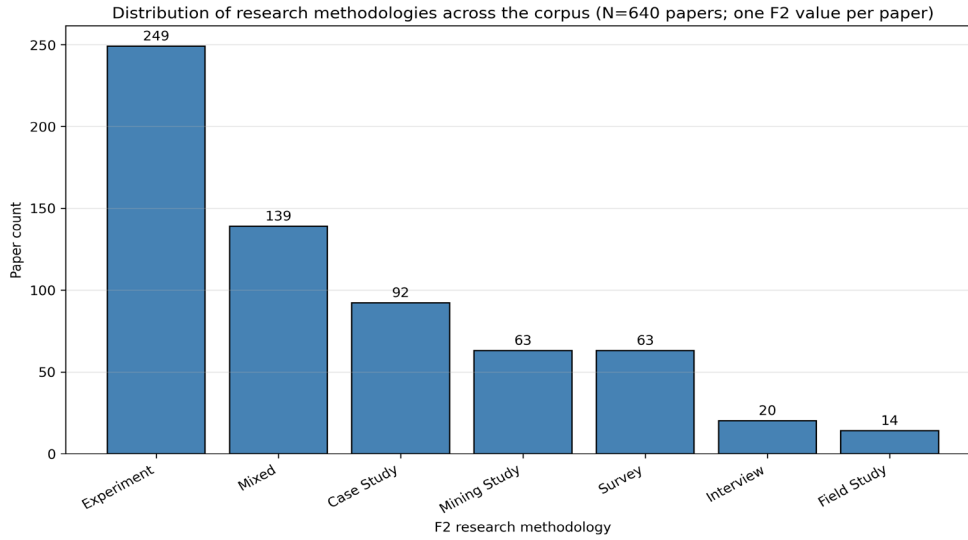


Figure 3: F2 research-methodology distribution for the 640-paper corpus.

F3 population coverage is partial. F3 shows that the empirical base is only partly informative about who uses AI-SE tools. Of the 640 included papers, 394 (61.6%) report a canonical population value, while 246 (38.4%) are missing or not applicable for population analysis. The populated subset is concentrated around three familiar study populations: students, professional software engineers, and mixed cohorts. By contrast, citizen developers and OSS contributors are rarely represented, with only 19 and 3 papers, respectively. This imbalance matters because citizen developers are central to the broader AI-SE adoption narrative, yet they remain weakly visible in the empirical corpus. Figure 3 illustrates this uneven coverage: the Citizen Developer and OSS Contributor panels are sparse compared with the student and professional-developer panels.

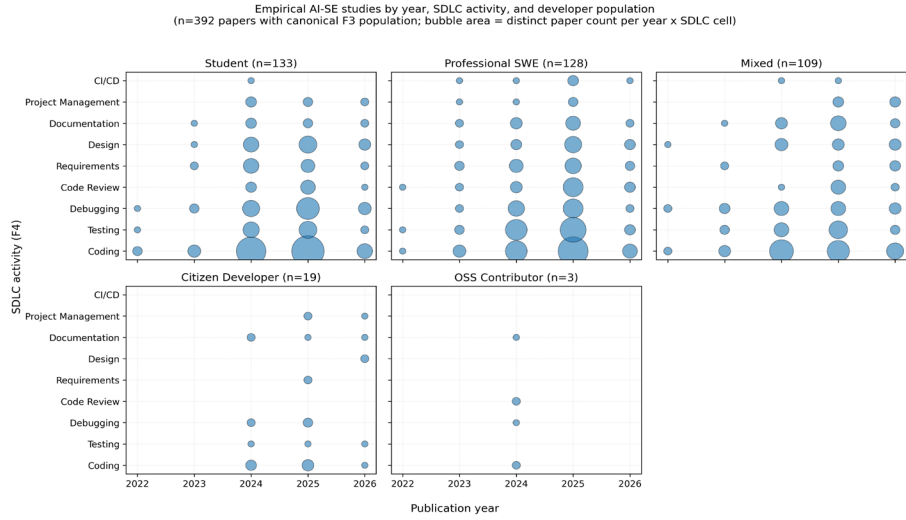


Figure 4: Year $\times$ SDLC activity $\times$ developer population bubble plot (n = 392 of the 394 canonical-F3 papers; 2 Mixed-population papers omitted because no F4 SDLC activity could be assigned). Bubble area = distinct paper count per year $\times$ SDLC cell.

F4 SDLC coverage is code-centric. The 640 papers produce 1,083 paper-SDLC activity bindings across the nine-activity taxonomy. Coding dominates (351 bindings), followed by Testing (189) and Debugging (165); together, these three activities account for 65.1% of all bindings. Coverage is much thinner for Project Management (37) and CI/CD (23), with Requirements (77) also trailing the code-centered activities. The SDLC-by-year heatmap in Figure 5 shows this persistent concentration over time. This uneven activity coverage provides the empirical backdrop for the capability-use gaps reported in RQ3.

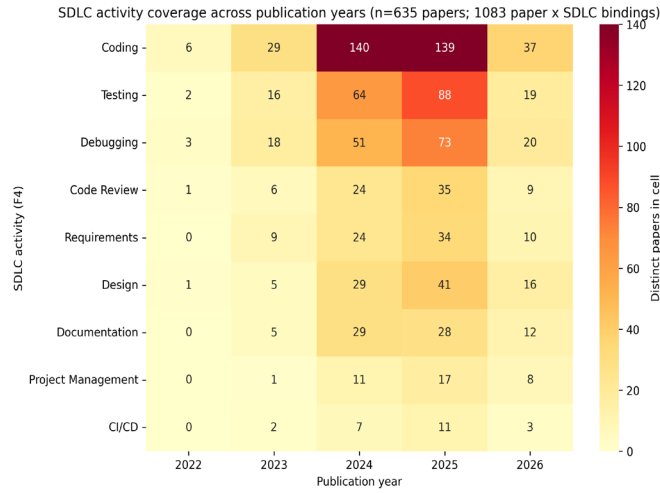
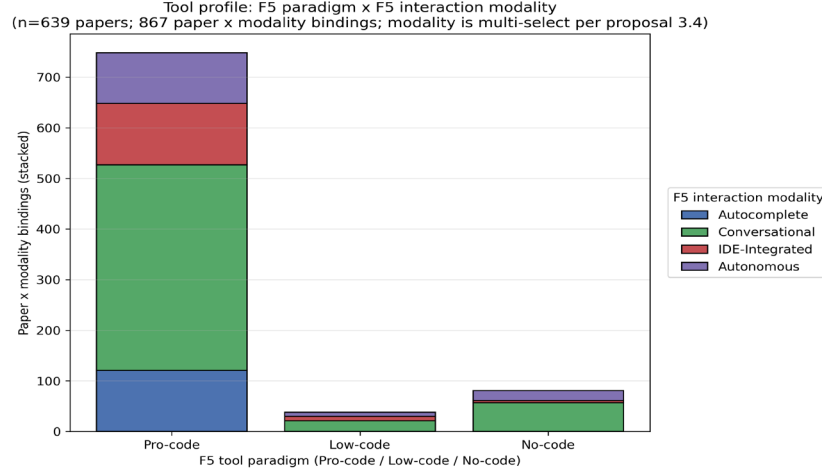


Figure 5: SDLC $\times$ year evidence heatmap. Each cell is the distinct-paper count for the given SDLC activity and publication year. Coding, Testing, and Debugging dominate; Requirements, Project Management, and CI/CD are thin across every year.

F5 pro-code tools predominate. All 640 papers have a coded development paradigm: 538 (84.1%) study pro-code tools, compared with 72 no-code papers (11.3%) and 30 low-code papers (4.7%). This skew means that paradigm-specific findings, especially for low-code and no-code settings, should be interpreted as directional rather than definitive. The stacked tool-profile in Figure 6 visualizes the paradigm and modality distribution; its modality stack uses 639 papers because one record has no plottable modality value.



Note: The stacked figure shows 639 papers because one paper lacks a plottable modality.

Figure 6 F5 tool profile. Paradigm (Pro-code, Low-code, No-code) on the horizontal axis; interaction-modality stacks (Autocomplete, Conversational, IDE-Integrated, Autonomous) within each paradigm bar.

4.2 RQ2 - Human–AI Interaction Mode Taxonomy

Delegation-depth axis. We define delegation depth as the extent to which responsibility for a software-engineering task shifts from human to the AI. The axis ranges from immediate, human-controlled assistance in Mode 1 to multi-step task execution in Mode 5, where the AI performs larger units of work and the human intervenes through steering, review, or final approval. The axis was pre-specified in the protocol; the modes placed along it were derived inductively from the corpus.

Five modes along the axis. The synthesis of 707 canonical labels produced a five-mode interaction taxonomy, summarized in Table 2.

Table 2: Five-mode interaction taxonomy summary

#	Mode	Canonical labels	Share (%)	Saturation
1	Inline Completion	4	0.6%	Stable; corpus-bias caveat (§4.2 & §6)
2	Conversational Prompting	69	9.8%	Stable
3	Visual / Declarative Composition	9	1.3%	Stable; paradigm-skew caveat
4	Review & Validation	82	11.6%	Stable
5	Delegated Task Execution	75	10.6%	Stable
r	Residuals (outcome / constraint / meta / affordance)	468	66.2%	Sub-structured (§4.2)

Mode 1 thinness reflects corpus visibility, not necessarily practitioner use. The four canonical Mode 1 labels fall well below the 15-label re-open trigger specified in the pre-registered synthesis workflow. We therefore interpret Mode 1 thinness as a property of the published and retained corpus, not as evidence of sparse practitioner use. Inline completion has been in production since 2021 and is no longer treated as a publication-worthy novelty in many venues.

Residuals form a coherent evidence-about-interaction layer rather than a sixth interaction mode. They capture outcomes, constraints, meta-observations, and tool affordances surrounding human-AI interaction, such as productivity effects, trust barriers, adoption patterns, and workflow-enabling features. We therefore report Residuals separately from the five-mode taxonomy.

Mode-level variation remains code-centered. Coding is the leading SDLC activity in every mode row, while Mode 4 is comparatively more visible in Code Review and Mode 5 has the strongest CI/CD footprint. Pro-code tools dominate across the delegation axis, with Mode 3 carrying the clearest low-code signal. Population-context coverage also remains concentrated in Professional SWE/Industry and Student/Education cells, leaving Citizen Developer and OSS Contributor settings too thin for mode-level prevalence claims.

Mode-layer saturation was reached. No new interaction modes emerged in the final 29 of 290 passage-bearing papers in processing order, so the five-mode partition is stable within this corpus. Canonical-label saturation was not reached: 63 new labels appeared in the final window. This is expected because canonical labels are finer-grained than modes.

4.3 RQ3 - Capability–evidence gap matrix

Research Question 3 examines how the 19 supply-side capabilities are represented in demand-side empirical evidence across the software development lifecycle. We constructed a 19×9 capability-by-SDLC matrix, yielding 171 possible intersections. Each cell counts distinct papers that report a given capability in each SDLC activity.

Using the primary gap rule defined in section 3.7, the 171 cells are divided into 117 adequately populated cells (68.4%), 42 gap cells (24.6%), and 12 empty cells (7.0%). A gap is a nonzero cell whose evidence count is at or below the 25th percentile of the nonzero cell distribution. In this corpus, $P_{25} = 2.0$; therefore, gap cells contain only one or two supporting papers. Empty cells have no supporting papers and are reported separately to distinguish under-studied intersections from intersections not observed in the corpus.

Evidence is concentrated in capability-SDLC pairings that align with the capability’s native use context. The densest cells are CAP_CODEGEN $\times$ Coding, CAP_TESTING $\times$ Testing, and CAP_DEBUGGING $\times$ Debugging. Off-diagonal evidence is much thinner, with gap and empty cells concentrated around early lifecycle activities such as Requirements and Design, and operational activities such as CI/CD and Project Management. Figure 7 visualizes this asymmetry, marking gap cells with red borders and empty cells with diagonal hatching.

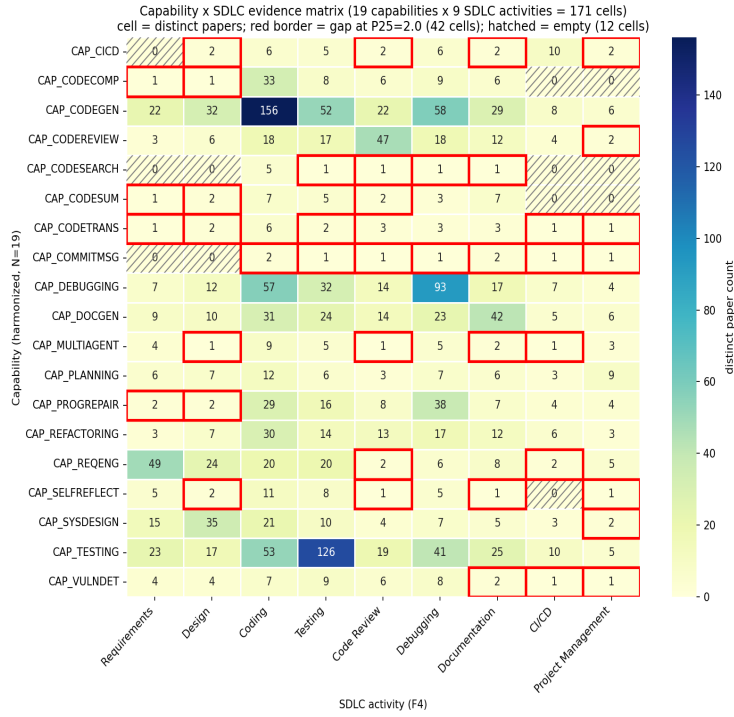


Figure 7: Capability × SDLC evidence matrix (19 capabilities × 9 SDLC activities = 171 cells)

Top-priority gaps cluster around capabilities that the supply-side literature already treats as central, but whose empirical use is unevenly documented across the SDLC. Table 3 shows that the highest-priority gaps are not marginal capabilities. They involve code completion, code translation, and self-reflective refinement: capabilities identified by all three reference surveys. The sparse cells occur when these capabilities move beyond their native coding context. Code completion has limited evidence in Requirements and Design; code translation is thin in Requirements, CI/CD, and Project Management; and self-reflective refinement is under-studied in Code Review, Documentation, and Project Management.

This pattern suggests that the main empirical gap is not whether these capabilities can be built, but whether and how they are used in adjacent development work. Mature AI-SE capabilities are well represented in coding-centered contexts, but less studied when they support upstream specification, design reasoning, review, documentation, or operational workflows. At the same time, not every sparse or empty off-diagonal cell should be interpreted as a research priority; some intersections may be structurally weak under the study’s definitions. The lower-ranked high-priority cells, including Multi-Agent Collaboration and Vulnerability Detection in CI/CD, Code Review, Design, and Project Management, reinforce the need to distinguish plausible capability transfer from architecturally inapplicable combinations.

Table 3: Top-13 prioritized capability × SDLC gaps (priority ≥ 1.00)

Rank	Capability ID	Label	SDLC Activity	Evidence	Priority	Gap Category
1	CAP_CODECOMP	Code Completion	Design	1	1.50	design_gap
2	CAP_CODECOMP	Code Completion	Requirements	1	1.50	design_gap
3	CAP_CODETRANS	Code Translation	CI/CD	1	1.50	organizational_readiness_barrier

Rank	Capability ID	Label	SDLC Activity	Evidence	Priority	Gap Category
4	CAP_CODETRANS	Code Translation	Project Management	1	1.50	organizational_readiness_barrier
5	CAP_CODETRANS	Code Translation	Requirements	1	1.50	design_gap
6	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Code Review	1	1.50	adoption_lag
7	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Documentation	1	1.50	adoption_lag
8	CAP_SELFREFLECT	Self-Reflection and Iterative Refinement	Project Management	1	1.50	adoption_lag
9	CAP_MULTIAGENT	Multi-Agent Collaboration	CI/CD	1	1.00	adoption_lag
10	CAP_MULTIAGENT	Multi-Agent Collaboration	Code Review	1	1.00	adoption_lag
11	CAP_MULTIAGENT	Multi-Agent Collaboration	Design	1	1.00	design_gap
12	CAP_VULNDET	Vulnerability Detection	CI/CD	1	1.00	organizational_readiness_barrier
13	CAP_VULNDET	Vulnerability Detection	Project Management	1	1.00	organizational_readiness_barrier

The gap categories clarify the type of empirical work that is missing. Most gaps reflect organizational readiness barriers, followed by design gaps and adoption lags. The strongest pattern is not scattered sparsity, but a structural imbalance: empirical AI-SE evidence remains concentrated around coding, testing, and debugging, while Requirements, Design, CI/CD, Documentation, and Project Management are thinner. Sensitivity checks preserve the P25 gap set across the examined threshold band, but paradigm-specific gap analysis remains underpowered because low-code and no-code submatrices contain many empty cells. These paradigm-specific results should therefore be read as descriptive signals rather than independent findings.

5 DISCUSSION

5.1 Supply-Side Saturation and Demand-Side Under-Evidence

The supply-side surveys describe a broad capability landscape [43, 68, 70] that now extends well beyond code generation. Examples include inline code completion, automated program repair, test generation, debugging, automated code review, code translation, documentation generation, CI/CD automation, task planning, multi-agent collaboration, and self-reflective agent refinement. Several of these capabilities are presented as part of agentic or lifecycle-spanning systems, yet the demand-side evidence remains concentrated around coding, testing, and debugging. For example, code completion is well established as an IDE-level coding capability, but our research finds only sparse evidence for its use in Requirements and Design. Code translation is discussed as a modernization and migration capability, but demand-side evidence is thin in Requirements, CI/CD, and Project Management. Self-reflective refinement is a prominent agentic capability in the supply-side literature, yet empirical evidence is sparse in Code Review, Documentation, and Project Management. Multi-agent collaboration and vulnerability detection show a similar pattern: they are visible in capability inventories but have limited demand-side evidence in CI/CD, Code Review, Design, and Project Management.

The RQ3 matrix makes this mismatch visible at the capability-SDLC level. Of the 171 capability-activity intersections, 42 are low-evidence gaps and 12 are empty cells. This does not mean that the corresponding tools do not exist. Rather, it means that published empirical evidence has not yet caught up with supply-side claims, especially in requirements, design, CI/CD, documentation, project management, low-code, and no-code settings. The distinction matters: supply-side surveys map technical possibilities, while this SMS maps where human use has been empirically observed.

The RQ2 interaction taxonomy shows the same asymmetry from another angle. Agentic systems are a prominent supply-side frontier, but Mode 5, Delegated Task Execution, accounts for 75 of 707 canonical labels. Conversely, Mode

1, Inline Completion, is represented by only four labels, even though inline completion is widely deployed in practice. We interpret this as a publication-visibility effect rather than evidence of low practitioner use. Mature tools are less likely to be studied as novel phenomena; while emerging agentic systems are more visible in current research.

This study therefore complements, rather than replaces, prior taxonomies and surveys. Treude and Gerosa [63] classify developer-AI interaction at the UI-mechanism level. The present SMS instead organizes interaction along a delegation-depth axis and links that interaction evidence to a capability-by-SDLC matrix. The result is a demand-side map: not what AI tools can do in principle, but where empirical studies show people using them.

5.2 Practical Implications

For **organizations**, the delegation-depth taxonomy offers a practical way to reason about adoption. Teams do not move from “no AI” to “autonomous agents” in one step. Mode 2 requires prompt formulation and iterative steering; Mode 4 requires review protocols and accountability; Mode 5 requires checkpoint design, trust calibration, and recovery procedures when the agent acts incorrectly. The taxonomy can therefore support a more concrete readiness assessment than generic AI-adoption scores. At the same time, the low visibility of Mode 1 in the corpus should not be read as a deployment sequence. Inline completion is likely underrepresented because it is already routine.

For **tool builders**, the strongest signal is that coding-stage features are no longer the only frontier. The highest-priority gaps involve capabilities that are already central in supply-side surveys but weakly evidenced in adjacent SDLC activities. Code completion is thin in Requirements and Design; code translation is thin in Requirements, CI/CD, and Project Management; and self-reflective refinement is thin in Code Review, Documentation, and Project Management. These are not all equally actionable, because some off-diagonal cells may be structurally weak. However, the broader implication is clear: future tool value depends not only on better coding assistance, but on better integration into planning, review, documentation, pipeline, and coordination work.

For **researchers**, the corpus points to a need for more in-situ empirical work. Experiments dominate the evidence base, while interviews and field studies remain comparatively rare. This is especially limiting for Mode 4 and Mode 5, where the central issues are trust, oversight, accountability, workflow fit, and organizational adoption. Although the studies remain useful, they should be complemented by field studies, longitudinal deployments, and practitioner interviews that capture how AI tools are actually embedded in team practice.

For **educators**, the taxonomy supports staged instruction in AI-assisted software engineering. Students can first learn low-delegation practices such as inline completion and conversational prompting, then move to review-and-validation tasks before being asked to delegate larger units of work. This progression emphasizes evaluation before delegation. It also responds to the risk identified in agentic settings: novice users may over-trust confident but incorrect AI output. The educational priority is therefore not tool exposure alone, but disciplined checkpointing, critique, and responsibility for final work products.

5.3 Future Research

The RQ3 matrix provides a basis for prioritizing future empirical work. Its value is not simply that it lists sparse cells, but that it shows where supply-side capability claims have limited demand-side evidence. The working agenda translates these gaps into study directions using Wieringa’s contribution-type vocabulary [76]. These directions should be read as priorities for empirical investigation, not as evidence that every sparse intersection is equally meaningful.

The first priority is to study how mature coding capabilities transfer into earlier or adjacent SDLC activities. Code completion and code translation are well represented in coding-centered contexts, but their use in requirements, design,

CI/CD, and migration project management remains thinly evidenced. Future studies should examine whether these tools preserve intent, support planning and design reasoning, and fit naturally into team workflows outside the editor.

A second priority is to evaluate agentic capabilities beyond coding tasks. Self-reflection and multi-agent orchestration are prominent in supply-side surveys, yet their demand-side evidence remains limited in code review, documentation, project management, design, and CI/CD. Future work should compare reflective or multi-agent systems against simpler baselines and measure not only task success, but also coordination quality, unsolicited actions, duplicated work, trust calibration, and accountability.

A third priority concerns operational and security workflows. Vulnerability detection and CI/CD-related capabilities are technically mature in many tool ecosystems, but the corpus contains limited evidence on how teams triage AI-generated security alerts, decide which alerts to act on, and integrate AI support into release pipelines. Field studies in industrial settings would be especially valuable here because the central barriers are likely organizational as much as technical.

A fourth priority is knowledge management across non-native SDLC activities. Code search, code summarization, and commit-message generation appear in scattered evidence outside their usual contexts. Rather than treating each sparse cell as a separate topic, future work could examine how teams use these capabilities across review, debugging, documentation, testing, and CI/CD decision points.

Finally, future studies should distinguish under-studied intersections from structurally weak or inapplicable ones. Some empty cells may indicate promising unexplored areas, such as summarization in CI/CD or code search for requirements discovery. Others, such as code completion in project management, are less meaningful under the study’s definitions. This distinction is important for turning the matrix into a credible research agenda rather than a mechanical list of low-count cells.

6 THREATS TO VALIDITY

We assessed trustworthiness using the qualitative-research criteria adapted for software engineering by Cruzes and Dybå [12, 13]: credibility, confirmability, dependability, and transferability.

Credibility. The main threat to credibility is that the study synthesizes a rapidly evolving field where terminology, tool capabilities, and adoption practices change quickly. To reduce post-hoc interpretation, the classification facets, interaction-mode synthesis procedure, and RQ3 gap rule were specified before analysis and implemented through auditable artifacts. RQ2 claims can be traced from interaction mode to canonical label, pass-1 code, passage identifier, and paper identifier. RQ3 gap claims are linked to the capability-by-SDLC matrix and to corresponding entries in the decision register. This traceability does not remove interpretive judgment, but it makes each judgment inspectable.

Confirmability. Researcher interpretation is a particular risk in thematic synthesis. We mitigated this by preserving source-to-claim links across the workflow. Generated CSV files include .meta.json sidecars recording scripts, inputs, and timestamps. Interpretive labels such as `organizational_readiness_barrier`, `design_gap`, and `adoption_lag` were recorded in the decision register rather than embedded directly in the matrix, preserving the separation between extracted evidence and synthesis-layer interpretation.

Dependability. The main dependability threat is whether the RQ2 taxonomy and RQ3 gap set would remain stable under reasonable analytic variation. For RQ2, mode-layer saturation was reached: no new interaction modes emerged in the final 29 of 290 passage-bearing papers. Canonical-label saturation was not reached, which is expected because canonical labels are more fine-grained than modes; therefore, the five-mode taxonomy is the dependable layer, not the full set of canonical labels. For RQ3, threshold sensitivity checks show that the 42-cell P25 gap set is preserved when the

threshold is varied across the examined percentile band. Higher thresholds add cells, but do not reverse the primary P25 result.

Transferability. The findings should be interpreted within clear corpus boundaries. The study covers peer-reviewed empirical papers from 2022 through the April 2026 search cutoff, using Scopus, ACM Digital Library, and backward snowballing from seed surveys. It excludes grey literature, non-English studies, benchmark-only model evaluations without human use, and secondary studies as primary evidence. The corpus is also pro-code dominant: low-code and no-code studies are comparatively sparse, and citizen-developer and OSS-contributor populations are underrepresented. RQ2 is further bounded by the 290 passage-bearing papers used for qualitative synthesis; papers without usable interaction passages still contribute to RQ1 and, where possible, RQ3, but not to mode derivation. These boundaries mean the results are strongest for published pro-code AI-SE evidence and more tentative for low-code, no-code, and broader practitioner settings.

Single-rater design. This SMS was conducted by a single rater with LLM assistance used for screening support, label proposal, and coding assistance. The rater retained final control over approvals, merges, splits, renamings, and borderline classifications, and relevant decisions were logged in `decision_register.csv`. This design improves auditability but does not provide inter-rater reliability evidence. A multi-rater replication or downstream practitioner validation would strengthen confidence in the taxonomy and in the interpretation of capability-SDLC gaps.

7 CONCLUSION

This systematic mapping study examined the demand side of empirical AI-SE research: how AI coding assistants, software engineering agents, and AI-augmented low-code and no-code platforms are reported as being used in software development work. Whereas recent supply-side surveys describe what these systems can do, this study maps where empirical evidence of human use has accumulated.

Across 640 primary empirical papers published between 2022 and April 2026 search cutoff, the study makes three contributions. First, it provides a five-facet evidence landscape showing that the corpus is recent, code-centered, and dominated by pro-code tools. Second, it derives a five-mode Human-AI Interaction Mode Taxonomy organized along a delegation-depth axis, from inline completion to delegated task execution. Third, it builds a 19×9 capability-by-SDLC matrix that compares harmonized supply-side capabilities with demand-side empirical evidence, distinguishing low-evidence gaps from empty intersections.

The main finding is a supply-demand asymmetry. AI-SE capability inventories are broad and fast-moving, but empirical evidence remains concentrated around coding, testing, and debugging. Evidence is thinner for requirements, design, documentation, CI/CD, project management, low-code, and no-code contexts. This does not imply that the missing intersections are all equally meaningful. Some are plausible but under-studied, such as self-reflective agents in code review or vulnerability detection in CI/CD. Others may be structurally weak under the study’s definitions, such as code completion in project management. The distinction between under-evidenced and inapplicable intersections is therefore essential for interpreting the gap matrix.

The study also shows that interaction mode, tool modality, and system capability should be treated as separate constructs. A tool may expose several modalities, support multiple capabilities, and still be used through different interaction modes depending on the task and user. The delegation-depth taxonomy helps make this distinction visible and provides a vocabulary for comparing studies that otherwise use inconsistent labels such as assistant, copilot, chatbot, and agent.

For practice, the findings suggest that organizations should evaluate AI adoption not only by tool availability, but by the level of delegation they are prepared to support. Tool builders should look beyond saturated coding-stage features toward integration with planning, review, documentation, pipeline, and coordination workflows. Researchers should prioritize field studies, interviews, and longitudinal evaluations in under-evidenced SDLC contexts. Educators can use the taxonomy to scaffold AI-SE instruction from low-delegation assistance toward higher-delegation tasks that require explicit review and accountability.

Future research should validate these patterns in practitioner settings, especially in low-code, no-code, operational, and early-lifecycle contexts that are underrepresented in the current corpus. To support auditability and replication, the study's evidence landscape, interaction taxonomy, capability-by-SDLC gap matrix, prioritized gaps, and future-research agenda are provided in the public replication package accompanying this article.

ACKNOWLEDGMENTS

The author would like to thank Professor Ta and Professor Zhu for their expert supervision, constructive feedback, and dedicated mentorship throughout the lifecycle of this project, which significantly shaped the direction of this research.

DECLARATION OF GENERATIVE AI USE

During the preparation of this work, the author used generative AI tools to support selected research and writing tasks. Gemini 3.1 Pro was used to assist with structured paper extraction, qualitative coding and label proposal, taxonomy-classification support, manuscript refinement, and figure generation. GPT-5.4 was used as an independent verifier to check extraction quality and validate analytical outputs. Claude Code was used as part of the repository execution environment to support code development, workflow implementation, and reproducibility checks. The author reviewed, verified, edited, and approved all AI-assisted outputs and takes full responsibility for the accuracy, integrity, and final content of the submitted manuscript.

REFERENCES

- [1] Matthew Oladeji Ajimati, Noel Carroll, and Mary Maher. 2025. Adoption of low-code and no-code development: A systematic literature review and future research agenda. *Journal of Systems and Software* 222, (April 2025), 112300. <https://doi.org/10.1016/j.jss.2024.112300>
- [2] Adam Alami, Victor Jensen, and Neil Ernst. 2025. Accountability in code review: The role of intrinsic drivers and the impact of llms. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3721127>
- [3] Farman Ali, Awais Ahmed, Mohammad Amin Alipour, and Hugo Terashima-Marin. 2025. Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model. *Journal of Computers in Education* (2025). <https://doi.org/10.1007/s40692-025-00375-w>
- [4] Nauman Bin Ali and Kai Petersen. 2014. Evaluating strategies for study selection in systematic literature studies. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, September 18, 2014. ACM, Torino Italy, 1–4. <https://doi.org/10.1145/2652524.2652557>
- [5] Aylton Almeida, Laerte Xavier, and Marco Tulio Valente. 2024. Automatic library migration using large language models: First results. In *Proceedings of the 18th ACM / IEEE international symposium on empirical software engineering and measurement (ESEM '24)*, 2024. . <https://doi.org/10.1145/3674805.3690746>
- [6] Eman Abdullah AlOmar, Anushkrishna Venkatakrishnan, Mohamed Wiem Mkaouer, Christian Newman, and Ali Ouni. 2024. How to refactor this code? An exploratory study on developer-ChatGPT refactoring conversations. In *Proceedings of the 21st international conference on mining software repositories (MSR 2024)*, 2024. . <https://doi.org/10.1145/3643991.3645081>
- [7] Urvi Awasthi, Lisa Krayner, and Daniel Sack. 2026. TurnStyle: a framework for analyzing human conversational behaviors to predict success in LLM-assisted tasks. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790459>

- [8] Leonardo Banh, Florian Holldack, and Gero Strobel. 2025. Copiloting the future: How generative AI transforms Software Engineering. *Information and Software Technology* 183, (July 2025), 107751. <https://doi.org/10.1016/j.infsof.2025.107751>
- [9] Jessica Y Bo, Majeed Kazemitabaar, Mengqing Deng, Michael Inzlicht, and Ashton Anderson. 2025. Invisible saboteurs: Sycophantic llms mislead novices in problem-solving tasks. In *ACM conference*, 2025. . <https://doi.org/10.1145/3772318.3791365>
- [10] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-Chi Cheung. 2025. A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair. *Automated Software Engineering* (2025). <https://doi.org/10.1007/s10515-025-00492-x>
- [11] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2024. Prompt sapper: a LLM-empowered production tool for building AI chains. *ACM Transactions on Software Engineering and Methodology* (2024). <https://doi.org/10.1145/3638247>
- [12] D. S. Cruzes and T. Dyba. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*, September 2011. IEEE, Banff, AB, 275–284. <https://doi.org/10.1109/ESEM.2011.36>
- [13] Daniela S. Cruzes, Tore Dybå, Per Runeson, and Martin Höst. 2015. Case studies synthesis: a thematic, cross-case, and narrative synthesis worked example. *Empir Software Eng* 20, 6 (December 2015), 1634–1665. <https://doi.org/10.1007/s10664-014-9326-8>
- [14] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub copilot AI pair programmer: Asset or liability? *Journal of Software and System* (2023). <https://doi.org/10.1016/j.jss.2023.111734>
- [15] Gerand Boy Elinzano, Michelle Ching, and Emmanuel Iluzada. 2024. Towards an AI-assisted software developer: The perspective on use of ChatGPT for programming in the philippines. In *Asian CHI symposium 2024*, 2024. . <https://doi.org/10.1145/3704611.3704621>
- [16] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent AI systems. In *CHI conference on human factors in computing systems*, 2025. . <https://doi.org/10.1145/3706598.3713581>
- [17] Leonardo Martín Esnaola, Hugo Dionisio Ramón, and Laura Cristina Lanzarini. 2026. Can generative AI bridge the gap? A quasi-experimental study of non-programmers with AI vs. programmers without AI. *Empirical Software Engineering* (2026). <https://doi.org/10.1007/s10664-026-10813-7>
- [18] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-based test-driven interactive code generation: User study and empirical evaluation. *arXiv* (2024). <https://doi.org/10.1109/tse.2024.3428972>
- [19] James Flamino, Mohammed Shahid Modi, Boleslaw K. Szymanski, Brendan Cross, and Colton Mikolajczyk. 2025. Testing the limits of large language models in debating humans. *Scientific Reports* (2025). <https://doi.org/10.1038/s41598-025-98378-1>
- [20] Claudia Flores-Saviaga, Benjamin V. Hanrahan, Kashif Imteyaz, Steven Clarke, and Saiph Savage*. 2025. The impact of generative AI coding assistants on developers who are visually impaired. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3714008>
- [21] Nat Friedman. 2021. Introducing GitHub Copilot: your AI pair programmer. *The GitHub Blog*. Retrieved April 7, 2026 from <https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/>
- [22] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. ChatGPT for vulnerability detection, classification, and repair: How far are we? In *APSEC*, 2023. . <https://doi.org/10.1109/apsec60848.2023.00085>
- [23] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3712005>
- [24] Leon Hannig, Annika Bush, Meltem Aksoy, Tim Trappen, Steffen Becker, and Greta Ontrup. 2026. Campus AI vs. commercial AI: Comparing how students and employees perceive their university’s LLM chatbot vs. ChatGPT. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790622>
- [25] Chieh-Yang Huang, Sanjana Gautam, Shannon McClellan Brooks, Ya-Fang Lin, Tiffany Kneareem, and Ting-Hao Kenneth Huang. 2024. Inspo: Writing with crowds alongside AI. In *arXiv*, 2024. . <https://doi.org/10.1145/3706599.3720193>
- [26] Qing Huang, Dianshu Liao, Zhenchang Xing, Zhiqiang Yuan, Qinghua Lu, Xiwei Xu, and Jiaxing Lu. 2022. SE factual knowledge in frozen giant code model: a study on FQN and its retrieval. *IEEE Transactions on Knowledge and Data Engineering* (2022). <https://doi.org/10.1109/tkde.2024.3436883>
- [27] Victor Vadmand Jensen, Adam Alami, Anders Rysholt Bruun, and John Stouby Persson. 2025. Managing expectations towards AI tools for software development: a multiple-case study. *Information Systems and e-Business Management* (2025). <https://doi.org/10.1007/s10257-025-00704-7>
- [28] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Bin Cui, and Peng Chen. 2025. SiriusBI: a comprehensive LLM-powered solution for data analytics in business intelligence. *Proceedings of the VLDB Endowment* (2025). <https://doi.org/10.14778/3750601.3750610>

- [29] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can ChatGPT support developers? An empirical evaluation of large language models for code generation. In *21st international conference on mining software repositories (MSR '24)*, 2024. . <https://doi.org/10.1145/3643991.3645074>
- [30] Bangyan Ju, Jin Yang, Tingting Yu, Tamerlan Abdullayev, Yuanyuan Wu, Dingbang Wang, and Yu Zhao. 2024. A study of using multimodal llms for non-crash functional bug detection in android apps. In *Apsec 2024*, 2024. . <https://doi.org/10.1109/apsec65559.2024.00017>
- [31] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-024-10594-x>
- [32] Kei Koyanagi, Dong Wang, Kotaro Noguchi, Masanari Kondo, Alexander Serebrenik, Yasutaka Kamei, and Naoyasu Ubayashi. 2024. Exploring the effect of multiple natural languages on code suggestion using GitHub copilot. In *21st international conference on mining software repositories*, 2024. . <https://doi.org/10.1145/3643991.3644917>
- [33] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI agents still need you: Findings from developer-agent collaborations in the wild. <https://doi.org/10.48550/arXiv.2506.12347>
- [34] Tzu-Sheng Kuo, Sophia Liu, Quan Ze Chen, Joseph Seering, Amy X. Zhang, Haiyi Zhu, and Kenneth Holstein. 2026. Botender: Supporting communities in collaboratively designing AI agents through case-based provocations. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3790500>
- [35] Thanh Le-Cong, Thanh-Dat Nguyen, Bach Le, and Toby Murray. 2023. Towards reliable evaluation of neural program repair with natural robustness testing. *ACM* (2023). <https://doi.org/10.1145/3716167>
- [36] Chunhua Liu, Hong Yi Lin, and Patanamon Thongtanunam. 2025. Too noisy to learn: Enhancing data quality for code review comment generation. In *MSR*, 2025. . <https://doi.org/10.1109/msr66628.2025.00043>
- [37] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2026. Large Language Model-Based Agents for Software Engineering: A Survey. *ACM Trans. Softw. Eng. Methodol.* (March 2026), 3796507. <https://doi.org/10.1145/3796507>
- [38] Michael Xieyang Liu, Frederick Liu, Alexander J. Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J. Cai. 2024. “We need structured output”: Towards user-centered constraints on large language model output. In *Chi ea '24*, 2024. . <https://doi.org/10.1145/3613905.3650756>
- [39] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2024. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. *IEEE Transactions on Software Engineering* (2024). <https://doi.org/10.1109/tse.2024.3392499>
- [40] Mauricio Monteiro, Bruno Castelo Branco, Samuel Silvestre, Guilherme Avelino, and Marco Tulio Valente. 2025. NoCodeGPT: a no-code interface for building web apps with language models. *arXiv* (2025). <https://doi.org/10.1002/spe.3432>
- [41] Michel Nass, Emil Alégroth, and Robert Feldt. 2023. Improving web element localization by using a large language model. *Software Testing, Verification and Reliability* (2023). <https://doi.org/10.1002/stvr.1893>
- [42] Abdullah Bin Nofal, Hassan Ali, Muhammad Hadi, Aizaz Ahmad, Adnan Qayyum, Aditya Johri, Ala Al-Fuqaha, and Junaid Qadir. 2025. AI-enhanced interview simulation in the metaverse: Transforming professional skills training through VR and generative conversational AI. *Computers and Education: Artificial Intelligence* (2025). <https://doi.org/10.1016/j.caeai.2024.100347>
- [43] Nesreen Otoum and Nuha Elkhaili. 2026. Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review. *IEEE Access* 14, (2026), 7443–7465. <https://doi.org/10.1109/ACCESS.2026.3652325>
- [44] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology* (2025). <https://doi.org/10.1145/3697010>
- [45] Eunil Park. 2025. Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10730-1>
- [46] Pat Pataranutaporn, Eunhae Lee, Judith Amores, and Pattie Maes. 2026. Personal validation effect in llms: Positive AI responses bias perceptions of validity, reliability, personalization, and usefulness of fictitious predictions. In *Proceedings of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772318.3791851>
- [47] Mauro Lourenço Pedra, Mônica Ferreira da Silva, and Leonardo Guerreiro Azevedo. 2021. DevOps Adoption: Eight Emergent Perspectives. <https://doi.org/10.48550/arXiv.2109.09601>
- [48] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. 2008. Systematic mapping studies in software engineering. In *Proceedings of the 12th international conference on Evaluation and Assessment in Software Engineering*. 68–77. <https://doi.org/10.5555/2227115.2227123>
- [49] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology* 64, (August 2015), 1–18. <https://doi.org/10.1016/j.infsof.2015.03.007>

- [50] Matei-Dan Popovici. 2024. ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course. *arXiv* (2024). <https://doi.org/10.1080/10447318.2023.2269006>
- [51] Piotr Przymus, Mikołaj Fejzer, Jakub Narębski, and Krzysztof Stencel. 2024. How I learned to stop worrying and love ChatGPT. In *21st international conference on mining software repositories (MSR '24)*, 2024. . <https://doi.org/10.1145/3643991.3645073>
- [52] Rachna Raj and Diego Elias Costa. 2024. The role of library versions in Developer-ChatGPT conversations. In *Proceedings of MSR '24: Proceedings of the 21st international conference on mining software repositories (MSR 2024)*, 2024. . <https://doi.org/10.1145/3643991.3645075>
- [53] Kent Ramchand, Mohan Baruwal Chhetri, and Ryszard Kowalczyk. 2021. Enterprise adoption of cloud computing with application portfolio profiling and application portfolio assessment. *J Cloud Comp* 10, 1 (January 2021), 1. <https://doi.org/10.1186/s13677-020-00210-w>
- [54] Charvi Rastogi, Marco Tulio Ribeiro, Nicholas King, Harsha Nori, and Saleema Amershi. 2023. Supporting human-AI collaboration in auditing llms with llms. In *AAAI/ACM conference on AI, ethics, and society (AIES '23)*, 2023. . <https://doi.org/10.1145/3600211.3604712>
- [55] Kevin Roitero, Dustin Wright, Michael Soprano, Isabelle Augenstein, and Stefano Mizzaro. 2025. Efficiency and effectiveness of LLM-based summarization of evidence in crowdsourced fact-checking. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval*, 2025. . <https://doi.org/10.1145/3726302.3729960>
- [56] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do LLMs consider security? an empirical study on responses to programming questions. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10658-6>
- [57] Raphael Serafini, Asli Yardim, and Alena Naiakshina. 2025. Exploring the impact of intervention methods on developers' security behavior in a manipulated ChatGPT study. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3713989>
- [58] Leuson Da Silva, Jordan Samhi, and Foutse Khomh. 2025. LLMs and stack overflow discussions: Reliability, impact, and challenges. *Journal of Systems and Software* (2025). <https://doi.org/10.1016/j.jss.2025.112541>
- [59] Vedant Das Swain, Qiuyue “Joy” Zhong, Jash Rajesh Parekh, Yechan Jeon, Roy Zimmermann, Mary P Czerwinski, Jina Suh, Varun Mishra, Koustuv Saha, and Javier Hernandez. 2025. AI on my shoulder: Supporting emotional labor in front-office roles with an LLM-based empathetic coworker. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3713705>
- [60] Florian Tambon, Arghavan Moradi-Dakheel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering* (2025). <https://doi.org/10.1007/s10664-025-10614-4>
- [61] Jiessie Tie, Bingsheng Yao, Tianshi Li, Hongbo Fang, Syed Ishtiaque Ahmed, Dakuo Wang, and Shurui Zhou. 2026. “Should I give up now?” investigating LLM pitfalls in software engineering. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3801972>
- [62] Utku Boran Torun, Mehmet Taha Demircan, Mahmut Furkan Gön, and Eray Tüzün. 2026. Past, present, and future of bug tracking in the generative AI era. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3806655>
- [63] Christoph Treude and Marco A. Gerosa. 2025. How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering. <https://doi.org/10.48550/arXiv.2501.08774>
- [64] Mario Truss. 2026. PersonaCite: VoC-grounded interviewable agentic synthetic AI personas for verifiable user and design research. In *Extended abstracts of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772363.3798543>
- [65] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT’s usage in open source projects: a mining-based study. In *21st international conference on mining software repositories (MSR '24)*, 2024. . <https://doi.org/10.1145/3643991.3644918>
- [66] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI conference on human factors in computing systems extended abstracts*, 2022. . <https://doi.org/10.1145/3491101.3519665>
- [67] Eszter Vigh, Freja Sunesen, and Louise Barkhuus. 2026. “AI does not understand the real world.”: AI Augmented Software Development. In *Extended abstracts of the 2026 CHI conference on human factors in computing systems*, 2026. . <https://doi.org/10.1145/3772363.3799079>
- [68] Huaning Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities. <https://doi.org/10.48550/arXiv.2508.11126>
- [69] Xingbo Wang, Janessa Griffith, Daniel A. Adler, Joey Castillo, Tanzeem Choudhury, and Fei Wang. 2025. Exploring personalized health support through data-driven, theory-guided llms: a case study in sleep health. In *CHI conference on human factors in computing systems (CHI '25)*, 2025. . <https://doi.org/10.1145/3706598.3713852>
- [70] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: survey, landscape, and vision. *Automated Software Engineering* 32, 2 (August 2025). <https://doi.org/10.1007/s10515-025-00544-2>

- [71] Miku Watanabe, Yutaro Kashiwa, Bin Lin, Toshiki Hirao, Ken'Ichi Yamaguchi, and Hajimu Iida. 2024. On the use of ChatGPT for code review: Do developers like reviews by ChatGPT? In *EASE 2024: 28th international conference on evaluation and assessment in software engineering*, 2024. . <https://doi.org/10.1145/3661167.3661183>
- [72] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2026. On the use of agentic coding: An empirical study of pull requests on GitHub. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3798166>
- [73] Justin D. Weisz, Shraddha Vijay Kumar, Michael Muller, Karen-Ellen Browne, Arielle Goldberg, Katrin Ellice Heintze, and Shagun Bajpai. 2025. Examining the use and impact of an AI code assistant on developer productivity and experience in the enterprise. In *Extended abstracts of the CHI conference on human factors in computing systems (CHI EA '25)*, 2025. . <https://doi.org/10.1145/3706599.3706670>
- [74] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically inspecting thousands of static bug warnings with large language model: How far are we? *ACM Trans. Knowl. Discov. Data.* (2024). <https://doi.org/10.1145/3653718>
- [75] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2024. Explaining explanations: An empirical study of explanations in code reviews. *ACM* (2024). <https://doi.org/10.1145/3708518>
- [76] Roel Wieringa, Neil Maiden, Nancy Mead, and Colette Rolland. 2006. Requirements engineering paper classification and evaluation criteria: a proposal and a discussion. *Requirements Eng* 11, 1 (March 2006), 102–107. <https://doi.org/10.1007/s00766-005-0021-6>
- [77] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *CHI conference on human factors in computing systems*, 2022. . <https://doi.org/10.1145/3491102.3517582>
- [78] Zhipeng Xue, Xiaoting Zhang, Zhipeng Gao, Xing Hu, Shan Gao, Xin Xia, and Shanping Li. 2026. Clean code, better models: Enhancing LLM performance with smell-cleaned dataset. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3793252>
- [79] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Fight fire with fire: How much can we trust ChatGPT on source code-related tasks? *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* (2024). <https://doi.org/10.1109/tse.2024.3492204>
- [80] Chi Zhang, Zhao Yang, Jiakuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. AppAgent: Multimodal agents as smartphone users. In *arXiv*, 2023. . <https://doi.org/10.1145/3706598.3713600>
- [81] Yuxia Zhang, Zhiqing Qiu, Klaas-Jan Stol, Wenhui Zhu, Jiaxin Zhu, Yingchen Tian, and Hui Liu. 2024. Automatic commit message generation: a critical review and directions for future work. *IEEE Transactions on Software Engineering* (2024). <https://doi.org/10.1109/tse.2024.3364675>
- [82] Xiyu Zhou, Peng Liang, Beiqi Zhang, Zengyang Li, Aakash Ahmad, Mojtaba Shahin, and Muhammad Waseem. 2024. Exploring the problems, their causes and solutions of AI pair programming: a study on GitHub and stack overflow. *Journal of Systems and Software* (2024). <https://doi.org/10.1016/j.jss.2024.112204>
- [83] Terry Yue Zhuo, Junda He, Jiamou Sun, Zhenchang Xing, David Lo, John Grundy, and Xiaoning Du. 2025. Identifying and mitigating API misuse in large language models. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* (2025). <https://doi.org/10.1109/tse.2026.3651566>

A APPENDICES

A.1 SDLC Activity × Tool Modality in the Ranked Full-Text Subset

Cross-tabulation of full-text-extracted papers from SCIS-ranked venues, indexed by SDLC activity (F4, rows) and tool modality (F5, columns).

Table 4: SDLC Activity × Tool Modality in the Ranked Full-Text Subset

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
Requirements	$n = 2$ [23, 27] Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	$n = 7$ [7, 11, 19, 23, 27, 34, 64] Awasthi et al. 2026 (10.1145/3772318.3790459). "TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks" Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains" Flamino et al. 2025 (10.1038/s41598-025-98378-1). "Testing the limits of large language models in debating humans" Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study" Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations" Truss 2026 (10.1145/3772363.3798543). "PersonaCite: VoC-Grounded Interviewable Agentic Synthetic AI Personas for Verifiable User and Design Research"	$n = 3$ [11, 23, 27] Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains" Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	$n = 1$ [19] Flamino et al. 2025 (10.1038/s41598-025-98378-1). "Testing the limits of large language models in debating humans"
Design	$n = 4$ [8, 23, 27, 67] Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting"	$n = 8$ [8, 11, 23, 27, 34, 38, 64, 67] Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting"	$n = 4$ [11, 23, 27, 67] Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	—

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>the future: How generative AI transforms Software Engineering</i>	<i>the future: How generative AI transforms Software Engineering</i>	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	
	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Vigh et al. 2026 (10.1145/3772363.3799079). ""AI does not understand the real world.". AI Augmented Software Development"	
	Vigh et al. 2026 (10.1145/3772363.3799079). ""AI does not understand the real world.". AI Augmented Software Development"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"		
		Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"		
		Liu et al. 2024 (10.1145/3613905.3650756). "We Need Structured Output: Towards User-centered Constraints on Large Language Model Output"		
		Truss 2026 (10.1145/3772363.3798543). "PersonaCite: VoC-Grounded Interviewable Agentic Synthetic AI Personas for Verifiable User and Design Research"		
		Vigh et al. 2026 (10.1145/3772363.3799079). ""AI does not understand the real world.". AI Augmented Software Development"		
Coding	<i>n = 17</i> [3, 8, 14, 18, 20, 23, 26, 27, 32, 45, 60, 66, 67, 73, 78, 82, 83] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model" Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	<i>n = 38</i> [3, 5–8, 10, 11, 15, 17, 20, 23, 24, 27–29, 34, 38–40, 42, 44, 45, 50–52, 56–58, 60–62, 65, 67, 69, 71, 73, 79, 81] AlOmar et al. 2024 (10.1145/3643991.3645081). "How to Refactor this Code? An Exploratory Study on Developer-ChatGPT Refactoring Conversations" Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model"	<i>n = 16</i> [3, 11, 14, 18, 20, 23, 26, 27, 32, 45, 60, 66, 67, 73, 82, 83] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and learning motivation through an extended technology acceptance model" Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains" Dakhel et al. 2023 (10.1016/j.jss.2023.111734). "GitHub Copilot AI pair programmer: Asset or Liability?"	<i>n = 5</i> [28, 62, 71, 78, 81] Jiang et al. 2025 (10.14778/3750601.3750610). "SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence" Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	Dakhel et al. 2023 (10.1016/j.jss.2023.111734). <i>"GitHub Copilot AI pair programmer: Asset or Liability?"</i>	Almeida et al. 2024 (10.1145/3674805.3690746). <i>"Automatic Library Migration Using Large Language Models: First Results"</i>	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). <i>"LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"</i>	Xue et al. 2026 (10.1145/3793252). <i>"Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"</i>
	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). <i>"LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"</i>	Awasthi et al. 2026 (10.1145/3772318.3790459). <i>"TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks"</i>	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). <i>"The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"</i>	Zhang et al. 2024 (10.1109/TSE.2024.3364675). <i>"Automatic Commit Message Generation: A Critical Review and Directions for Future Work"</i>
	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). <i>"The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"</i>	Banh et al. 2025 (10.1016/j.infsof.2025.107751). <i>"Copiloting the future: How generative AI transforms Software Engineering"</i>	Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i>	
	Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i>	Cao et al. 2025 (10.1007/s10515-025-00492-x). <i>"A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair"</i>	Huang et al. 2022 (10.1109/TKDE.2024.3436883). <i>"SE Factual Knowledge in Frozen Giant Code Model: A Study on FQN and its Retrieval"</i>	
	Huang et al. 2022 (10.1109/TKDE.2024.3436883). <i>"SE Factual Knowledge in Frozen Giant Code Model: A Study on FQN and its Retrieval"</i>	Cheng et al. 2024 (10.1145/3638247). <i>"Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"</i>	Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	Elinzano et al. 2024 (10.1145/3704611.3704621). <i>"Towards an AI-Assisted Software Developer: The Perspective on Use of ChatGPT for Programming in the Philippines"</i>	Koyanagi et al. 2024 (10.1145/3643991.3644917). <i>"Exploring the Effect of Multiple Natural Languages on Code Suggestion Using GitHub Copilot"</i>	
	Koyanagi et al. 2024 (10.1145/3643991.3644917). <i>"Exploring the Effect of Multiple Natural Languages on Code Suggestion Using GitHub Copilot"</i>	Esnaola et al. 2026 (10.1007/s10664-026-10813-7). <i>"Can generative AI bridge the gap? A quasi-experimental study of non-programmers with AI vs. programmers without AI"</i>	Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>	
	Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). <i>"The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"</i>	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	
	Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i>	Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i>	Vaithilingam et al. 2022 (10.1145/3491101.3519665). <i>"Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models"</i>	
	Vaithilingam et al. 2022 (10.1145/3491101.3519665). <i>"Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models"</i>	Hannig et al. 2026 (10.1145/3772318.3790622). <i>"Campus AI vs. Commercial AI: Comparing How Students and Employees Perceive their University's LLM Chatbot vs. ChatGPT"</i>	Vigh et al. 2026 (10.1145/3772363.3799079). <i>"'AI does not understand the real world.'. AI Augmented Software Development"</i>	
	Vigh et al. 2026 (10.1145/3772363.3799079). <i>"'AI does not understand the real world.'. AI Augmented Software Development"</i>		Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code"</i>	

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"</i>	Jensen et al. 2025 (10.1007/s10257-025-00704-7). <i>"Managing expectations towards AI tools for software development: a multiple-case study"</i>	<i>Assistant on Developer Productivity and Experience in the Enterprise"</i>	
	Xue et al. 2026 (10.1145/3793252). <i>"Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"</i>	Jiang et al. 2025 (10.14778/3750601.3750610). <i>"SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence"</i>	Zhou et al. 2024 (10.1016/j.jss.2024.112204). <i>"Exploring the Problems, their Causes and Solutions of AI Pair Programming: A Study on GitHub and Stack Overflow"</i>	
	Zhou et al. 2024 (10.1016/j.jss.2024.112204). <i>"Exploring the Problems, their Causes and Solutions of AI Pair Programming: A Study on GitHub and Stack Overflow"</i>	Jin et al. 2024 (10.1145/3643991.3645074). <i>"Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation"</i>	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	
	Zhuo et al. 2025 (10.1109/tse.2026.3651566). <i>"Identifying and Mitigating API Misuse in Large Language Models"</i>	Kuo et al. 2026 (10.1145/3772318.3790500). <i>"Botender: Supporting Communities in Collaboratively Designing AI Agents through Case-Based Provocations"</i>		
		Liu et al. 2024 (10.1109/TSE.2024.3392499). <i>"No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT"</i>		
		Liu et al. 2024 (10.1145/3613905.3650756). <i>"We Need Structured Output: Towards User-centered Constraints on Large Language Model Output"</i>		
		Monteiro et al. 2025 (10.1002/spe.3432). <i>"NoCodeGPT: A No-Code Interface for Building Web Apps with Language Models"</i>		
		Nofal et al. 2025 (10.1016/j.caeai.2024.100347). <i>"AI-enhanced interview simulation in the metaverse: Transforming professional skills training through VR and generative conversational AI"</i>		
		Ouyang et al. 2025 (10.1145/3697010). <i>"An Empirical Study of the Non-Determinism of ChatGPT in Code Generation"</i>		
		Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i>		
		Popovici 2024 (10.1080/10447318.2023.2269006). <i>"ChatGPT in the classroom. Exploring its potential and</i>		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		limitations in a Functional Programming course."		
		Przymus et al. 2024 (10.1145/3643991.3645073). "How I Learned to Stop Worrying and Love ChatGPT"		
		Raj et al. 2024 (10.1145/3643991.3645075). "The role of library versions in Developer-ChatGPT conversations"		
		Sajadi et al. 2025 (10.1007/s10664-025-10658-6). "Do LLMs consider security? an empirical study on responses to programming questions"		
		Serafini et al. 2025 (10.1145/3706598.3713989). "Exploring the Impact of Intervention Methods on Developers' Security Behavior in a Manipulated ChatGPT Study"		
		Silva et al. 2025 (10.1016/j.jss.2025.112541). "LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges"		
		Tambon et al. 2025 (10.1007/s10664-025-10614-4). "Bugs in large language models generated code: an empirical study"		
		Tie et al. 2026 (10.1145/3801972). "'Should I Give Up Now?'" Investigating LLM Pitfalls in Software Engineering"		
		Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"		
		Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"		
		Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"		
		Wang et al. 2025 (10.1145/3706598.3713852). "Exploring Personalized Health Support through Data-Driven, Theory-Guided LLMs: A Case Study in Sleep Health"		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
Testing		Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"		
		Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"		
		Yu et al. 2024 (10.1109/TSE.2024.3492204). "Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"		
		Zhang et al. 2024 (10.1109/TSE.2024.3364675). "Automatic Commit Message Generation: A Critical Review and Directions for Future Work"		
	<i>n</i> = 7 [8, 18, 20, 23, 27, 45, 73]	<i>n</i> = 15 [5, 8, 11, 20, 23, 27, 34, 45, 50, 54, 62, 65, 71, 73, 79]	<i>n</i> = 7 [11, 18, 20, 23, 27, 45, 73]	<i>n</i> = 6 [30, 41, 55, 62, 71, 80]
	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Almeida et al. 2024 (10.1145/3674805.3690746). "Automatic Library Migration Using Large Language Models: First Results"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Ju et al. 2024 (10.1109/APSEC65559.2024.00017). "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps"
	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). "LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Fakhoury et al. 2024 (10.1109/TSE.2024.3428972). "LLM-based Test-driven Interactive Code Generation: User Study and Empirical Evaluation"	Nass et al. 2023 (10.1002/stvr.1893). "Improving web element localization by using a large language model"
	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Roitero et al. 2025 (10.1145/3726302.3729960). "Efficiency and Effectiveness of LLM-Based Summarization of Evidence in Crowdsourced Fact-Checking"
	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"
	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Zhang et al. 2023 (10.1145/3706598.3713600). "AppAgent: Multimodal Agents as Smartphone Users"
	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	Kuo et al. 2026 (10.1145/3772318.3790500). "Botender: Supporting Communities in	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	<i>Assistant on Developer Productivity and Experience in the Enterprise"</i>	<i>Collaboratively Designing AI Agents through Case-Based Provocations"</i> Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i> Popovici 2024 (10.1080/10447318.2023.2269006). <i>"ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course."</i> Rastogi et al. 2023 (10.1145/3600211.3604712). <i>"Supporting Human-AI Collaboration in Auditing LLMs with LLMs"</i> Torun et al. 2026 (10.1145/3806655). <i>"Past, Present, and Future of Bug Tracking in the Generative AI Era"</i> Tufano et al. 2024 (10.1145/3643991.3644918). <i>"Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"</i> Watanabe et al. 2026 (10.1145/3798166). <i>"On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"</i> Weisz et al. 2025 (10.1145/3706599.3706670). <i>"Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"</i> Yu et al. 2024 (10.1109/TSE.2024.3492204). <i>"Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"</i>	<i>Assistant on Developer Productivity and Experience in the Enterprise"</i>	
Code Review	$n = 5$ [8, 23, 45, 67, 78] Banh et al. 2025 (10.1016/j.infsof.2025.107751). <i>"Copiloting the future: How generative AI transforms Software Engineering"</i>	$n = 11$ [2, 8, 22–24, 45, 50, 51, 65, 67, 75] Alami et al. 2025 (10.1145/3721127). <i>"Accountability in Code Review: The Role of Intrinsic Drivers and the Impact of LLMs"</i> Banh et al. 2025 (10.1016/j.infsof.2025.107751). <i>"Copiloting</i>	$n = 3$ [23, 45, 67] Gao et al. 2025 (10.1145/3712005). <i>"The Current Challenges of Software Engineering in the Era of Large Language Models"</i> Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants</i>	$n = 4$ [31, 36, 74, 78] Kang et al. 2025 (10.1007/s10664-024-10594-x). <i>"Explainable automated debugging via large language model-driven scientific debugging"</i> Liu et al. 2025 (10.1109/MSR66628.2025.00043). <i>"Too Noisy</i>

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Xue et al. 2026 (10.1145/3793252). "Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"	the future: How generative AI transforms Software Engineering" Fu et al. 2023 (10.1109/apsec60848.2023.00085). "ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?" Gao et al. 2025 (10.1145/3712005). "The Current Challenges of Software Engineering in the Era of Large Language Models" Hannig et al. 2026 (10.1145/3772318.3790622). "Campus AI vs. Commercial AI: Comparing How Students and Employees Perceive their University's LLM Chatbot vs. ChatGPT" Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models" Popovici 2024 (10.1080/10447318.2023.2269006). "ChatGPT in the classroom. Exploring its potential and limitations in a Functional Programming course." Przymus et al. 2024 (10.1145/3643991.3645073). "How I Learned to Stop Worrying and Love ChatGPT" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development" Widyasari et al. 2024 (10.1145/3708518). "Explaining Explanations: An Empirical Study of Explanations in Code Reviews"	among South Korean Industry Developers: A survey case study with large language models" Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	To Learn: Enhancing Data Quality for Code Review Comment Generation" Wen et al. 2024 (10.1145/3653718). "Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?" Xue et al. 2026 (10.1145/3793252). "Clean Code, Better Models: Enhancing LLM Performance with Smell-Cleaned Dataset"
Debugging	$n = 6$ [3, 8, 20, 45, 60, 83] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and	$n = 24$ [3, 7–11, 15, 16, 20, 22, 29, 39, 45, 52, 54, 56, 58, 60–62, 65, 71, 77, 79] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and	$n = 6$ [3, 11, 20, 45, 60, 83] Ali et al. 2025 (10.1007/s40692-025-00375-w). "Adoption of AI-coding assistants in programming education: exploring trust and	$n = 7$ [16, 30, 31, 35, 62, 71, 74] Epperson et al. 2025 (10.1145/3706598.3713581). "Interactive

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
	learning motivation through an extended technology acceptance model"	learning motivation through an extended technology acceptance model"	learning motivation through an extended technology acceptance model"	Debugging and Steering of Multi-Agent AI Systems"
	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Awasthi et al. 2026 (10.1145/3772318.3790459). "TurnStyle: A Framework for Analyzing Human Conversational Behaviors to Predict Success in LLM-Assisted Tasks"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Ju et al. 2024 (10.1109/APSEC65559.2024.00017). "A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps"
	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Banh et al. 2025 (10.1016/j.infsof.2025.107751). "Copiloting the future: How generative AI transforms Software Engineering"	Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"	Kang et al. 2025 (10.1007/s10664-024-10594-x). "Explainable automated debugging via large language model-driven scientific debugging"
	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Bo et al. 2025 (10.1145/3772318.3791365). "Invisible Saboteurs: Sycophantic LLMs Mislead Novices in Problem-Solving Tasks"	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Le-Cong et al. 2023 (10.1145/3716167). "Towards Reliable Evaluation of Neural Program Repair with Natural Robustness Testing"
	Tambon et al. 2025 (10.1007/s10664-025-10614-4). "Bugs in large language models generated code: an empirical study"	Cao et al. 2025 (10.1007/s10515-025-00492-x). "A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair"	Tambon et al. 2025 (10.1007/s10664-025-10614-4). "Bugs in large language models generated code: an empirical study"	Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"
	Zhuo et al. 2025 (10.1109/tse.2026.3651566). "Identifying and Mitigating API Misuse in Large Language Models"	Cheng et al. 2024 (10.1145/3638247). "Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains"	Zhuo et al. 2025 (10.1109/tse.2026.3651566). "Identifying and Mitigating API Misuse in Large Language Models"	Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"
		Elinzano et al. 2024 (10.1145/3704611.3704621). "Towards an AI-Assisted Software Developer: The Perspective on Use of ChatGPT for Programming in the Philippines"		Wen et al. 2024 (10.1145/3653718). "Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We?"
		Epperson et al. 2025 (10.1145/3706598.3713581). "Interactive Debugging and Steering of Multi-Agent AI Systems"		
		Flores-Saviaga et al. 2025 (10.1145/3706598.3714008). "The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired"		
		Fu et al. 2023 (10.1109/apsec60848.2023.00085). "ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?"		
		Jin et al. 2024 (10.1145/3643991.3645074). "Can ChatGPT Support Developers? An		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		<i>Empirical Evaluation of Large Language Models for Code Generation"</i> Liu et al. 2024 (10.1109/TSE.2024.3392499). <i>"No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT"</i> Park 2025 (10.1007/s10664-025-10730-1). <i>"Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"</i> Raj et al. 2024 (10.1145/3643991.3645075). <i>"The role of library versions in Developer-ChatGPT conversations"</i> Rastogi et al. 2023 (10.1145/3600211.3604712). <i>"Supporting Human-AI Collaboration in Auditing LLMs with LLMs"</i> Sajadi et al. 2025 (10.1007/s10664-025-10658-6). <i>"Do LLMs consider security? an empirical study on responses to programming questions"</i> Silva et al. 2025 (10.1016/j.jss.2025.112541). <i>"LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges"</i> Tambon et al. 2025 (10.1007/s10664-025-10614-4). <i>"Bugs in large language models generated code: an empirical study"</i> Tie et al. 2026 (10.1145/3801972). <i>"Should I Give Up Now?" Investigating LLM Pitfalls in Software Engineering"</i> Torun et al. 2026 (10.1145/3806655). <i>"Past, Present, and Future of Bug Tracking in the Generative AI Era"</i> Tufano et al. 2024 (10.1145/3643991.3644918). <i>"Unveiling ChatGPTâ€™s Usage in Open Source Projects: A Mining-based Study"</i> Watanabe et al. 2026 (10.1145/3798166). <i>"On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"</i> Wu et al. 2022 (10.1145/3491102.3517582). <i>"AI Chains: Transparent and Controllable</i>		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		Human-AI Interaction by Chaining Large Language Model Prompts"		
		Yu et al. 2024 (10.1109/TSE.2024.3492204). "Fight Fire with Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?"		
Documentation	<i>n</i> = 4 [27, 45, 67, 73]	<i>n</i> = 9 [15, 27, 29, 45, 65, 67, 72, 73, 81]	<i>n</i> = 5 [25, 27, 45, 67, 73]	<i>n</i> = 2 [72, 81]
	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Elinzano et al. 2024 (10.1145/3704611.3704621). "Towards an AI-Assisted Software Developer: The Perspective on Use of ChatGPT for Programming in the Philippines"	Huang et al. 2024 (10.1145/3706599.3720193). "Inspo: Writing with Crowds Alongside AI"	Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"
	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Zhang et al. 2024 (10.1109/TSE.2024.3364675). "Automatic Commit Message Generation: A Critical Review and Directions for Future Work"
	Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	Jin et al. 2024 (10.1145/3643991.3645074). "Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation"	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"
	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"	Park 2025 (10.1007/s10664-025-10730-1). "Continuance use of AI coding assistants among South Korean Industry Developers: A survey case study with large language models"	Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"	Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"
		Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"		
		Vigh et al. 2026 (10.1145/3772363.3799079). "'AI does not understand the real world.': AI Augmented Software Development"		
		Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"		
		Weisz et al. 2025 (10.1145/3706599.3706670). "Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise"		
		Zhang et al. 2024 (10.1109/TSE.2024.3364675). "Automatic		

F4 SDLC Activity	Autocomplete	Conversational	IDE-Integrated	Autonomous
		<i>Commit Message Generation: A Critical Review and Directions for Future Work"</i>		
Project	<i>n</i> = 1 [27]	<i>n</i> = 5 [27, 46, 59, 62, 65]	<i>n</i> = 1 [27]	<i>n</i> = 1 [62]
Management	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study" Pataranutaporn et al. 2026 (10.1145/3772318.3791851). "Personal Validation Effect in LLMs: Positive AI Responses Bias Perceptions of Validity, Reliability, Personalization, and Usefulness of Fictitious Predictions" Swain et al. 2025 (10.1145/3706598.3713705). "AI on My Shoulder: Supporting Emotional Labor in Front-Office Roles with an LLM-based Empathetic Coworker" Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study"	Jensen et al. 2025 (10.1007/s10257-025-00704-7). "Managing expectations towards AI tools for software development: a multiple-case study"	Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era"
CI/CD	—	<i>n</i> = 3 [62, 65, 72] Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Tufano et al. 2024 (10.1145/3643991.3644918). "Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"	—	<i>n</i> = 2 [62, 72] Torun et al. 2026 (10.1145/3806655). "Past, Present, and Future of Bug Tracking in the Generative AI Era" Watanabe et al. 2026 (10.1145/3798166). "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub"